

Using and Creating APIs

As a software engineer, you are likely familiar with building applications that interact with various external services and data sources. One of the most common methods for communication and integration is through HTTP APIs (Application Programming Interfaces). HTTP APIs provide a standardized way for applications to exchange data and functionality over the internet.

This chapter will introduce you to the world of HTTP APIs and explore how you can leverage them in your software development projects. We will cover the fundamental concepts, techniques, and best practices for effectively working with HTTP APIs.

An HTTP API allows two software systems to communicate and exchange information using the Hypertext Transfer Protocol (HTTP). It enables your application to make requests to an API server and receive responses in a structured format, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language).

Using HTTP APIs offers a range of benefits for software engineers. It allows you to leverage external services and data sources, enabling your application to access functionality or retrieve valuable information from third-party systems. This opens up opportunities for integration with popular platforms, social media networks, payment gateways, geolocation services, and much more.

Throughout this chapter, we will explore various aspects of working with HTTP APIs, including:

- **API endpoints and methods:** Understanding how to interact with an API involves identifying the available endpoints and the supported methods, such as GET, POST, PUT, DELETE, and so on. We will discuss how to construct API requests and handle different response formats.
- **Authentication and authorization:** Many APIs require authentication to ensure secure access and protect sensitive data. We will delve into different authentication mechanisms, including API keys, tokens, OAuth, and other authentication protocols commonly used in API integrations.
- **Request parameters and payloads:** APIs often accept additional parameters or payloads to customize the request or send data for processing. We will explore how to pass query parameters, request headers, and request bodies when interacting with APIs.
- **Error handling and status codes:** Learning how to handle errors and interpret status

codes returned by APIs is crucial for building robust and resilient applications. We will discuss common status codes and best practices for handling various scenarios gracefully.

- Rate limiting and throttling: Many APIs impose restrictions on the number of requests you can make within a given timeframe to prevent abuse and ensure fair usage. We will cover techniques for handling rate limiting and implementing efficient strategies to manage API quotas.
- API documentation and testing: Proper documentation is essential for understanding an API's capabilities, endpoints, and expected behavior. We will explore how to read and interpret API documentation, as well as techniques for testing and validating API integrations.

By mastering the art of using HTTP APIs, you will expand your development toolkit and gain the ability to seamlessly integrate your applications with external services, leverage their functionalities, and build powerful, interconnected systems.

So, let's dive into the world of HTTP APIs and uncover the endless possibilities they offer for enhancing your software engineering projects.

1.1 Setting up an API

Create a file called `server.js`. This will be a basic webserver that we will use to test our an API. This is called a *RESTful API*. Write the following code to create a basic server for our API.

```
1  const express = require("express");
2
3  const app = express();
4  const PORT = 3000;
5
6  const student = {
7    student_id: 42,
8    name: "John Smith",
9    major: "Computer Science",
10   credits_completed: 88
11 };
12
13 app.get("/students/42", (req, res) => {
14   res.json(student);
15 });
16
17 app.listen(PORT, () => {
18   console.log(`API running at http://localhost:${PORT}`);
19 });
```

Then, run the following

```
1 npm init -y
2 npm install express
3 node server.js
```

This will create a new server, along with a ton of dependency files that javascript and node automatically install as part of your server.

1.1.1 What is an endpoint?

An *endpoint* is a URL exposed by the API. It usually is able to display a page or data based on the request of the endpoint.

We can navigate to the endpoint `GET /students/42`, where `students` is the resource that usually maps to a database, and `42` is the id of a student in the database. To retrieve this student from the database, we can navigate to

```
1 GET http://localhost:3000/students/42
```

which is an endpoint from our API. This will output

```
1 {
2   "student_id": 42,
3   "name": "John Smith",
4   "major": "Computer Science",
5   "credits_completed": 88
6 }
```

Note that this is done in json format, common for backend data parsing.

Each endpoint combines a resource path with an HTTP method to complete an action.

This leads us to the idea of *CRUD operations*. CRUD, which stands for CREATE, REMOVE, UPDATE, DELETE, are a form of outlining an API's routes. Calling a route that aims to alter should follow one of the CRUD operations. GET does not have an operation since it is *read-only*.

For a database representing students (assume more than just one), we would want to have routes that can complete each CRUD operation.

Before an API can receive data from a client, Express must parse JSON request bodies.

HTTP Method	Purpose	CRUD Op	Example
GET	Fetch Data	N/A	Get student information
PUT	Update Data	UPDATE	Fully update student
PATCH	Update section of Data	UPDATE	Update a student's major
POST	Create Data	CREATE	Create a new student
DELETE	Remove Data	DELETE	Remove a student after graduation

Add the following line after creating the application.

```
1 app.use(express.json());
```

This middleware automatically converts incoming JSON into a JavaScript object that can be accessed through `req.body`.

1.1.2 Reading all resources

Most APIs provide a route for retrieving an entire collection. Change the `students` array to look like the following.

```
1 const students = [
2   {
3     student_id: 42,
4     name: "John Smith",
5     major: "Computer Science",
6     credits_completed: 88
7   },
8   {
9     student_id: 43,
10    name: "Jane Doe",
11    major: "Mathematics",
12    credits_completed: 61
13  }
14 ];
15
16 app.get("/students", (req, res) => {
17   res.json(students);
18 });
```

The endpoint

```
1 GET /students
```

returns every student currently stored in the collection. Notice that we did not include an id. Now, there are more than one possible ids, so we need to make a general route that takes in an id of a student to fetch

```

1 app.get("/students/:id", (req, res) => {
2   // use the id that is passed in the route
3   const id = Number(req.params.id);
4   // find a student from our students array
5   const student = students.find((s) => s.student_id === id);
6
7   // if the id is not found, we assume the student is not in the array
8   if (!student) {
9     return res.status(404).json({ error: "Student not found" });
10  }
11
12  res.json(student);
13 });

```

Start the server. Now, `http://localhost:3000/students` returns the entire students array, while `http://localhost:3000/students/43` only returns information about Jane Doe.

Routing to `http://localhost:3000/students/19` is invalid, so it returns

```

1 {"error": "Student not found"}

```

1.2 Requests with cURL

Curl, sometimes stylized as cURL or curl, is a command-line based resource used for sending API and HTTP requests for web server data.

The simplest curl command

```

1 curl http://example.com

```

which will return the html source code of the website <http://example.com>.

```

1 <!doctype html>
2 <html lang="en">
3
4 <head>
5   <title>Example Domain</title>
6   <link rel="icon" href="data:,">
7   <meta name="viewport" content="width=device-width, initial-scale=1">

```

```

8     <style>
9         body {
10             background: #eee;
11             width: 60vw;
12             margin: 15vh auto;
13             font-family: system-ui, sans-serif
14         }
15
16         h1 {
17             font-size: 1.5em
18         }
19
20         div {
21             opacity: 0.8
22         }
23
24         a:link,
25         a:visited {
26             color: #348
27         }
28     </style>
29 </head>
30
31 <body>
32     <div>
33         <h1>Example Domain</h1>
34         <p>This domain is for use in documentation examples without needing
35         ↪ permission. Avoid use in operations.</p>
36         <p><a href="https://iana.org/domains/example">Learn more</a></p>
37     </div>
38 </body>
39 </html>

```

FIXME expand here

1.3 Creating resources with POST

That's all great, but we can put cURL to work on much more laborious tasks. Going back to our created api example, we can post, update, and delete resources using cURL.

A POST request creates a new resource. Unlike GET requests, POST requests send data inside the request body.

```

1 app.post("/students", (req, res) => {
2     const student = req.body;
3

```

```
4 // basic validation: ensure a body is received
5 if (!student) {
6   return res.status(400).json({ error: "Invalid student data" });
7 }
8
9 students.push(student);
10
11 // status 201 means that the student was pushed successfully
12 res.status(201).json(student);
13 };
```

Notice the use of

```
1 req.body
```

which contains the JSON sent by the client.

HTTP requests are composed of headers and a body. Headers are metadata such as Content-Type (most commonly application/json) or Authorization that describe how the request should be handled. The body is the payload that carries actual data, like the student record being created by a POST request.

In Express, request headers are available on `req.headers`. For JSON payloads, middleware like `express.json()` parses the body so that the data is available on `req.body`.

A template for sending json in the body looks something like the following:

```
1 curl -X METHOD URL \
2     -H "Header: Value" \
3     -d 'JSON DATA'
```

To send a request body, include the body data in the HTTP request and set the appropriate Content-Type header. For example, with curl:

```
1 curl -X POST http://localhost:3000/students \
2     -H "Content-Type: application/json" \
3     -d '{
4       "student_id": 44,
5       "name": "Jerry Doe",
6       "major": "Computer Science",
7       "credits_completed": 23
8     }'
```

This can be broken down as:

- `curl`: The command-line tool for making HTTP requests.
- `-X POST`: Use the POST HTTP method.
- `http://localhost:3000/students`: The endpoint being contacted. Must exist in our server routes.
- `-H "Content-Type: application/json"`: Tell the server the request body contains JSON.
- `-d '{ ... }'`: Send this JSON as the request body.

The request body in this example is the JSON string passed with `-d`. Express will parse that JSON into `req.body` when `express.json()` middleware is enabled.

Note that our `server.js` `students` array will not update with Jerry Doe, because the server only stores the `students` array upon initialization, but we can see our now three students by running a GET request on `/students`

```
1  [  
2    {  
3      "student_id": 42,  
4      "name": "John Smith",  
5      "major": "Computer Science",  
6      "credits_completed": 88  
7    },  
8    {  
9      "student_id": 43,  
10     "name": "Jane Doe",  
11     "major": "Mathematics",  
12     "credits_completed": 61  
13   },  
14   {  
15     "student_id": 44,  
16     "name": "Jerry Doe",  
17     "major": "Computer Science",  
18     "credits_completed": 23  
19   }  
20 ]
```

FIXME talk about API keys, API requests, how it interacts with HTTP requests, why keys should not be shared
FIXME postman

[org/](#)) for more details.

Answers to Exercises



INDEX

CRUD operations, [3](#)

endpoint, [3](#)

HTTP, [1](#)

read-only, [3](#)

RESTful API, [2](#)

Web APIs, [1](#)